

Advanced Agent Patterns: ReAct, Reflection and Memory

Advanced Agent Patterns: ReAct, Reflection and Memory

Welcome to today's class. Over the next two hours, we will go deep into three powerful patterns that separate basic LLM applications from truly capable AI agents.

Here is what we are covering today:

1. ReAct Pattern: How agents interleave reasoning with actions to solve complex tasks
2. Reflection Pattern: How agents critique and improve their own outputs
3. Memory Systems: How agents remember, retrieve, and use information across interactions

Why does this matter?

Think about how products like Google Search, GitHub Copilot, or Amazon Alexa work behind the scenes. They do not just generate text. They reason about what to do, take actions (search the web, run code, query databases), observe the results, and adjust. These are agent patterns in production at massive scale.

By the end of this session, you will understand the mechanics behind these patterns and be able to build them using LangGraph with Gemini for agentic workflows, and the raw Gemini SDK for reflection loops.

Let us start by setting up our environment.

Code

```
!pip install -q langgraph langchain-groq
```

Output

```
[?25l  ????????????????????????????????????????????????????????????? 0.0/137.5 kB ? eta -:-:--
???????????????????????????????????????????????????????????? 137.5/137.5 kB 5.3 MB/s eta 0:00:00
```

Code

```
import os
from google.colab import userdata
os.environ["GROQ_API_KEY"]=userdata.get("GROQ_API_KEY")
```

Code

```
from langchain_groq import ChatGroq
import os
llm=ChatGroq(model="qwen/qwen3-32b",temperature=0,api_key=os.environ["GROQ_API_KEY"])
print(llm.invoke("Say Groq ready").content)
```

Output

```
<think>
Okay, the user said "Say Groq ready". I need to respond appropriately. First, I should
acknowledge their request. They might be testing if I can respond to specific commands or just
want a confirmation. Since the user is asking me to say "Groq ready", I should check if
there's any context I'm missing. Maybe they're referring to a specific system or model called
```

Groq? But I'm Qwen, so I need to clarify if they want me to simulate Groq's response or just say that I'm ready. Since the instruction is straightforward, the best approach is to comply and say "Groq ready" as requested. However, I should also add a note explaining that I'm Qwen and not Groq, to avoid confusion. That way, the user knows I'm following their instruction but also clarifying my identity. Let me make sure the response is clear and helpful.

Code

```
from langchain_groq import ChatGroq
import os
llm=ChatGroq(model="qwen/qwen3-32b",temperature=0,api_key=os.environ["GROQ_API_KEY"])
print(llm.invoke("Groq reflection ready").content)
```

Output

<think>

Okay, the user mentioned "Groq reflection ready." I need to figure out what they're referring to. Groq is a company known for their high-performance AI chips, right? Maybe they're talking about using Groq's technology for some kind of reflection process. But what does "reflection ready" mean in this context?

Reflection in AI usually refers to the ability of a system to analyze its own processes or decisions. So perhaps the user is asking about integrating Groq's hardware with reflective AI systems. But I'm not sure if Groq has specific products related to reflection. Maybe they're using Groq's chips to enhance the computational power needed for reflection tasks.

Alternatively, "reflection ready" could be a term from a specific framework or project. The user might be referring to a setup where Groq's hardware is prepared to handle reflective computations. I should check if there's any existing documentation or projects that combine Groq with reflection.

Wait, maybe the user is confused and meant something else. Let me think. Groq's main product is the TPU-like chip for AI inference. If they're using Groq for AI models that require reflection, like self-improving systems, then the hardware needs to support that. But I don't recall Groq mentioning reflection in their specs.

Another angle: the user might be asking about preparing Groq's hardware for reflective processes, like debugging or self-assessment in AI models. But again, I'm not sure if Groq has specific tools for that. It's possible the user is mixing terms or referring to a niche application.

I should ask for clarification. Maybe they want to know how to set up Groq's hardware for reflective AI tasks, or if Groq supports such features. Alternatively, they might be referring to a different technology altogether. Without more context, it's hard to pinpoint. The best approach is to request more details to provide an accurate response.

</think>

It seems like you're referring to **Groq**, a company known for its high-performance AI chips and accelerators, and the term **"reflection ready"** might relate to a specific use case or framework. However, the exact context isn't entirely clear. Here's how I can help:

Possible Interpretations:

1. **Groq Hardware for AI Reflection**:

If you're asking about using Groq's hardware (e.g., their **GroqChip**) to enable **AI reflection** (e.g., self-improving systems, meta-learning, or introspective AI), I can provide insights into how Groq's architecture supports high-speed computation for such tasks.

2. **"Reflection Ready" Framework**:

If "reflection ready" refers to a specific framework, tool, or project (e.g., a system requiring introspective capabilities), I can help clarify how Groq's hardware integrates with

such workflows.

3. **Debugging/Analysis Tools**:

If you're referring to **debugging** or **introspection tools** for AI models (e.g., analyzing model behavior), Groq's ecosystem might have tools or partnerships to support this.

How Can I Assist?

To provide a precise answer, could you clarify:

- What you mean by **"reflection ready"** (e.g., self-improving AI, debugging, introspection)?
- Are you working with a specific framework (e.g., TensorFlow, PyTorch, or a custom system)?
- Are you asking about Groq's hardware capabilities, software tools, or integration examples?

Let me know, and I'll tailor the response!

If you see both LangGraph ready! and Gemini SDK ready! above, we are all set. We are using two interfaces today for a deliberate reason:

- * LangGraph is purpose-built for agent loops (ReAct). It manages the cycle of reasoning, tool calling, and observation automatically.

- * Groq via LangChain gives us full control over individual LLM calls (Reflection). No framework magic, just clean function calls.

Let us dive in.

ReAct Pattern: Concept Deep Dive

[TIME: 0:10 to 0:30]

The ReAct Pattern: Reason + Act

ReAct stands for Reasoning and Acting. It was introduced in a 2022 paper by Yao et al. and has become one of the foundational patterns for building LLM agents.

The Core Idea

Most LLMs, when prompted, either:

- Reason about a problem (Chain-of-Thought), OR
- Act by calling tools or generating outputs

ReAct combines both into a single interleaved loop. The agent cycles through three steps repeatedly:

This loop repeats until the agent has enough information to produce a final answer.

Analogy: The Senior Engineer at Amazon

Imagine a senior engineer at Amazon gets paged at 2 AM because checkout is failing for European customers.

Here is how they work through it:

Thought 1: Checkout failures in Europe. Could be a payment gateway issue, a regional CDN problem, or a database timeout. Let me check the payment service logs first.

Action 1: Opens CloudWatch, filters payment service logs for EU region.

Observation 1: Payment service is healthy. No errors in the last hour.

Thought 2: Payment is fine. Let me check if the EU load balancer is routing correctly.

Action 2: Checks the load balancer dashboard.

Observation 2: The EU-west-1 load balancer shows 90% 5xx errors starting 45 minutes ago.

Thought 3: Found it. The EU load balancer is failing. Let me check what changed recently.

Action 3: Checks the deployment history.

Observation 3: A config change was pushed to EU-west-1 at 1:15 AM.

Final Answer: The checkout failure is caused by a bad config deployment to EU-west-1. Rolling back.

This is exactly how a ReAct agent operates. It does not try to answer everything in one shot. It thinks, acts, observes, and refines its understanding iteratively.

A Concrete ReAct Trace

Let us look at what a ReAct trace looks like for an e-commerce support scenario:

Question: A customer says their order 1005 has not arrived. They want to know what they ordered and whether the items are still available for a replacement.

...

Thought 1: I need to first look up order 1005 to see its status and contents.

Action 1: `lookup_order(1005)`

Observation 1: Order 1005 - Status: Delayed, Items: [Wireless Mouse, USB-C Hub],

Customer: Jane Smith

Thought 2: The order is delayed. The customer wants to know if replacements are available. Let me check inventory for Wireless Mouse first.

Action 2: `check_inventory(Wireless Mouse)`

Observation 2: Wireless Mouse - In Stock: 45 units, Warehouse: Seattle-W2

Thought 3: Wireless Mouse is available. Now let me check USB-C Hub.

Action 3: `check_inventory(USB-C Hub)`

Observation 3: USB-C Hub - In Stock: 3 units, Warehouse: Austin-W1

Thought 4: Both items are in stock, though USB-C Hub is running low.

I can now provide a complete answer.

Final Answer: Order 1005 (Wireless Mouse and USB-C Hub) is currently delayed.

Both items are available for replacement. Note that USB-C Hub has only 3 units left, so a replacement should be processed quickly.

...

Notice how the agent does not guess at inventory levels. It retrieves real data at each step and reasons about what to do next based on what it found.

How ReAct Compares to Other Patterns

You have already seen Chain-of-Thought (CoT) prompting. Let us compare it with two other patterns.

Key insight: CoT is pure reasoning with no external actions. Plan-and-Solve creates a full plan upfront and then executes it. ReAct is adaptive, meaning it decides what to do next based on what it just learned. This makes ReAct more robust for real-world tasks where the path to the answer is not known in advance.

QUIZ 1

Look at the following lines from an agent trace and type your answer in chat. Classify each line as Thought (T), Action (A), or Observation (O).

Format your answer like: (or whatever you think it is).

Answer to Quiz 1:

- * Line 1: Thought (reasoning about what to do)
- * Line 2: Action (calling a tool)
- * Line 3: Observation (result from the tool)
- * Line 4: Thought (reasoning about the observation and planning the next action)

Great. Now let us build this pattern with real code.

ReAct Pattern: Runnable Demo

[TIME: 0:30 to 0:50]

Building a ReAct Agent with Custom Tools

We will build a ReAct agent that acts as an e-commerce support assistant. It has access to three tools that query our (simulated) company systems:

1. `lookup_order`: Gets order details by order ID
2. `get_product_details`: Gets product information by product name
3. `check_inventory`: Checks stock levels for a product

All three tools use hardcoded dictionaries, so there are no external API calls and zero chance of network failures. Let us build the data and tools.

Code


```

# Our simulated e-commerce database
ORDERS_DB = {
  "1001": {
    "order_id": "1001",
    "customer": "Alice Johnson",
    "items": ["Laptop Pro 15", "Wireless Mouse"],
    "status": "Delivered",
    "order_date": "2024-11-20",
    "delivery_date": "2024-11-25",
    "total": 1349.98
  },
  "1002": {
    "order_id": "1002",
    "customer": "Bob Martinez",
    "items": ["USB-C Hub", "Mechanical Keyboard"],
    "status": "Shipped",
    "order_date": "2024-12-01",
    "estimated_delivery": "2024-12-07",
    "total": 214.98
  },
  "1003": {
    "order_id": "1003",
    "customer": "Carol Lee",
    "items": ["Noise Cancelling Headphones", "Laptop Pro 15", "Phone Stand"],
    "status": "Processing",
    "order_date": "2024-12-05",
    "total": 1479.97
  },
  "1004": {
    "order_id": "1004",
    "customer": "David Kim",
    "items": ["Wireless Mouse", "Phone Stand"],
    "status": "Cancelled",
    "order_date": "2024-11-15",
    "cancellation_reason": "Customer requested cancellation",
    "total": 74.98
  },
  "1005": {
    "order_id": "1005",
    "customer": "Eva Chen",
    "items": ["Mechanical Keyboard", "Monitor 27 inch"],
    "status": "Delayed",
    "order_date": "2024-11-28",
    "delay_reason": "Warehouse backlog",
    "total": 579.98
  }
}

PRODUCTS_DB = {
  "Laptop Pro 15": {
    "name": "Laptop Pro 15",
    "price": 1299.99,
    "category": "Computers",
    "warranty": "24 months",
    "description": "15-inch professional laptop with 16GB RAM and 512GB SSD",
    "rating": 4.7
  },
  "Wireless Mouse": {
    "name": "Wireless Mouse",
    "price": 49.99,
    "category": "Accessories",
    "warranty": "12 months",
    "description": "Ergonomic wireless mouse with 3 DPI settings",
    "rating": 4.3
  }
}

```

Output

```
Database loaded: 5 orders, 7 products, 7 inventory records
```

Now let us wrap these databases into LangChain-compatible tools. We use the `@tool` decorator, which is the simplest way to create tools in LangChain. The docstring of each function is critical because the LLM reads it to understand when and how to use the tool.

Code

```

from langchain_core.tools import tool
@tool
def lookup_order(order_id: str) -> str:
    """Look up an order by its order ID. Returns order details including
    customer name, items ordered, order status, dates, and total amount.
    Use this when you need to find information about a specific order."""
    order = ORDERS_DB.get(order_id)
    if not order:
        return f"Error: No order found with ID '{order_id}'. Valid order IDs are:
        {list(ORDERS_DB.keys())}"
    result = f"Order {order['order_id']}: \n"
    result += f"  Customer: {order['customer']} \n"
    result += f"  Items: {', '.join(order['items'])} \n"
    result += f"  Status: {order['status']} \n"
    result += f"  Order Date: {order['order_date']} \n"
    result += f"  Total: ${order['total']} \n"
    if 'delivery_date' in order:
        result += f"  Delivery Date: {order['delivery_date']} \n"
    if 'estimated_delivery' in order:
        result += f"  Estimated Delivery: {order['estimated_delivery']} \n"
    if 'delay_reason' in order:
        result += f"  Delay Reason: {order['delay_reason']} \n"
    if 'cancellation_reason' in order:
        result += f"  Cancellation Reason: {order['cancellation_reason']} \n"
    return result
@tool
def get_product_details(product_name: str) -> str:
    """Get detailed information about a product by its name. Returns price,
    category, warranty period, description, and customer rating.
    Use this when you need to know product specifications, pricing, or warranty info."""
    product = PRODUCTS_DB.get(product_name)
    if not product:
        available = list(PRODUCTS_DB.keys())
        return f"Error: Product '{product_name}' not found. Available products: {available}"
    result = f"Product: {product['name']} \n"
    result += f"  Price: ${product['price']} \n"
    result += f"  Category: {product['category']} \n"
    result += f"  Warranty: {product['warranty']} \n"
    result += f"  Description: {product['description']} \n"
    result += f"  Rating: {product['rating']}/5.0 \n"
    return result
@tool
def check_inventory(product_name: str) -> str:
    """Check the current inventory level for a product. Returns the number
    of units in stock and the warehouse location.
    Use this when you need to know if a product is available or how many units remain."""
    inventory = INVENTORY_DB.get(product_name)
    if not inventory:
        available = list(INVENTORY_DB.keys())
        return f"Error: Product '{product_name}' not found in inventory. Available products:
        {available}"
    status = "In Stock" if inventory["in_stock"] > 0 else "Out of Stock"
    low_stock_warning = " (LOW STOCK)" if 0 < inventory["in_stock"] <= 5 else ""
    result = f"Inventory for {product_name}: \n"
    result += f"  Status: {status}{low_stock_warning} \n"
    result += f"  Units Available: {inventory['in_stock']} \n"
    result += f"  Warehouse: {inventory['warehouse']} \n"
    return result
# Quick test to make sure all tools work
print("Testing lookup_order:")
print(lookup_order.invoke({"order_id": "1001"}))
print("\nTesting get_product_details:")
print(get_product_details.invoke({"product_name": "Wireless Mouse"}))

```

Output

```
Testing lookup_order:  
Order 1001:  
Customer: Alice Johnson  
Items: Laptop Pro 15, Wireless Mouse  
Status: Delivered  
Order Date: 2024-11-20  
Total: $1349.98  
Delivery Date: 2024-11-25  
Testing get_product_details:  
Product: Wireless Mouse  
Price: $49.99  
Category: Accessories  
Warranty: 12 months  
Description: Ergonomic wireless mouse with 3 DPI settings  
Rating: 4.3/5.0  
Testing check_inventory:  
Inventory for Noise Cancelling Headphones:  
Status: Out of Stock  
Units Available: 0  
Warehouse: Austin-W1
```

All three tools are working. Notice a few design choices:

- * Each tool returns a formatted string, not a dictionary. This makes it easier for the LLM to read and reason about the results.
- * Each tool has error handling that tells the LLM what valid inputs look like. This helps the agent self-correct if it guesses a wrong product name or order ID.
- * The docstrings are detailed. The LLM uses these to decide which tool to call and when.

Now let us create the ReAct agent.

Code

```
from langchain_groq import ChatGroq
from langgraph.prebuilt import create_react_agent
import os
llm=ChatGroq(model="qwen/qwen3-32b",temperature=0,api_key=os.environ["GROQ_API_KEY"])
tools=[lookup_order,get_product_details,check_inventory]
agent=create_react_agent(model=llm,tools=tools,prompt="You are a helpful e-commerce customer support agent.")
print("ReAct agent created.")
```

Output

```
ReAct agent created.
```

Error Output

```
/tmp/ipykernel_16995/1586022328.py:6: LangGraphDeprecatedSinceV10: create_react_agent has been moved to `langchain.agents`. Please update your import to `from langchain.agents import
```

`create_agent``. Deprecaded in LangGraph V1.0 to be removed in V2.0.
`agent=create_react_agent(model=llm,tools=tools,prompt="You are a helpful e-commerce customer`

```
support_agent.")
```

Let us create a helper function that runs the agent and prints each step clearly so we can see the full ReAct trace.

Code

```

def run_agent(query: str):
    """Run the ReAct agent and display the full trace of steps."""
    print("=" * 70)
    print(f"QUERY: {query}")
    print("=" * 70)
    step_count = 0
    tool_call_count = 0
    for step in agent.stream(
        {"messages": [{"role": "user", "content": query}]},
        stream_mode="updates"
    ):
        for node_name, node_output in step.items():
            messages = node_output.get("messages", [])
            for msg in messages:
                msg_type = msg.__class__.__name__
                if msg_type == "AIMessage":
                    # The agent's reasoning or final answer
                    if msg.content:
                        step_count += 1
                        print(f"\n[Step {step_count} - Agent Reasoning]")
                        print(msg.content)
                    # Check for tool calls
                    if hasattr(msg, "tool_calls") and msg.tool_calls:
                        for tc in msg.tool_calls:
                            tool_call_count += 1
                            print(f"\n[Step {step_count} - Tool Call #{tool_call_count}]")
                            print(f"  Tool: {tc['name']}")
                            print(f"  Input: {tc['args']}")
                elif msg_type == "ToolMessage":
                    print(f"\n[Observation from {msg.name}]")
                    print(f"  {msg.content}")
            print(f"\n{'=' * 70}")
    print(f"Trace complete. Tool calls made: {tool_call_count}")
    print("=" * 70)

```


Query 1: Simple Single-Tool Lookup

Let us start with a straightforward question that requires just one tool call.

Code

```
run_agent("What is the status of order 1002? Who placed it and when will it arrive?")
```

Output

```
=====
QUERY: What is the status of order 1002? Who placed it and when will it arrive?
=====
[Step 0 - Tool Call #1]
Tool: lookup_order
Input: {'order_id': '1002'}
[Observation from lookup_order]
Order 1002:
Customer: Bob Martinez
Items: USB-C Hub, Mechanical Keyboard
Status: Shipped
Order Date: 2024-12-01
Total: $214.98
Estimated Delivery: 2024-12-07
[Step 1 - Agent Reasoning]
The status of order 1002 is **Shipped**. It was placed by **Bob Martinez** on December 1,
2024. The estimated delivery date is **December 7, 2024**. Let me know if you need further
details!
=====
Trace complete. Tool calls made: 1
=====
```

Notice the agents behavior:

1. It recognized this is an order lookup question
2. It called with the correct order ID
3. It read the observation and formulated a friendly answer

This is a single-hop ReAct trace. One thought, one action, one observation, and a final answer. Now let us increase the complexity.

Query 2: Multi-Tool Reasoning

This question requires the agent to use multiple tools and connect information across them.

Code

```
run_agent(  
    "I placed order 1002. Can you tell me the price and warranty "  
    "information for each item in my order?"  
)
```

Output

=====

QUERY: I placed order 1002. Can you tell me the price and warranty information for each item in my order?

=====

[Step 0 - Tool Call #1]

Tool: lookup_order

Input: {'order_id': '1002'}

[Observation from lookup_order]

Order 1002:

Customer: Bob Martinez

Items: USB-C Hub, Mechanical Keyboard

Status: Shipped

Order Date: 2024-12-01

Total: \$214.98

Estimated Delivery: 2024-12-07

[Step 0 - Tool Call #2]

Tool: get_product_details

Input: {'product_name': 'USB-C Hub'}

[Step 0 - Tool Call #3]

Tool: get_product_details

Input: {'product_name': 'Mechanical Keyboard'}

[Observation from get_product_details]

Product: USB-C Hub

Price: \$89.99

Category: Accessories

Warranty: 12 months

Description: 7-in-1 USB-C hub with HDMI, USB 3.0, and SD card reader

Rating: 4.5/5.0

[Observation from get_product_details]

Product: Mechanical Keyboard

Price: \$124.99

Category: Accessories

Warranty: 24 months

Description: RGB mechanical keyboard with Cherry MX Blue switches

Rating: 4.6/5.0

[Step 1 - Agent Reasoning]

[Output continues below...]

```
Here are the details for your order (1002):
**USB-C Hub**
- Price: $89.99
- Warranty: 12 months
**Mechanical Keyboard**
- Price: $124.99
- Warranty: 24 months
**Order Total:** $214.98
Let me know if you'd like additional details about your order!
=====
Trace complete. Tool calls made: 3
=====
```

This is where ReAct gets interesting. The agent had to:

1. First look up order 1002 to find out what items were ordered
2. Then call for each item separately
3. Finally synthesize all the information into a coherent response

The agent decided what to do at each step based on what it learned in the previous step. It could not have planned all the tool calls upfront because it did not know the items until after the first lookup.

Query 3: Complex Multi-Step Reasoning

Now let us give the agent a question that requires reasoning across all three tools.

Code

```

run_agent(
    "I have order 1003. Can you check which of my items are currently "
    "in stock and which ones might be hard to replace if something goes wrong? "
    "Also, what is the total warranty coverage I have?"
)

```

Output

```

=====
QUERY: I have order 1003. Can you check which of my items are currently in stock and which
ones might be hard to replace if something goes wrong? Also, what is the total warranty
coverage I have?
=====
[Step 0 - Tool Call #1]
Tool: lookup_order
Input: {'order_id': '1003'}
[Observation from lookup_order]
Order 1003:
Customer: Carol Lee
Items: Noise Cancelling Headphones, Laptop Pro 15, Phone Stand
Status: Processing
Order Date: 2024-12-05
Total: $1479.97
[Step 0 - Tool Call #2]
Tool: check_inventory
Input: {'product_name': 'Noise Cancelling Headphones'}
[Step 0 - Tool Call #3]
Tool: check_inventory
Input: {'product_name': 'Laptop Pro 15'}
[Step 0 - Tool Call #4]
Tool: check_inventory
Input: {'product_name': 'Phone Stand'}
[Observation from check_inventory]
Inventory for Noise Cancelling Headphones:
Status: Out of Stock
Units Available: 0
Warehouse: Austin-W1
[Observation from check_inventory]
Inventory for Laptop Pro 15:
Status: In Stock
Units Available: 12
Warehouse: Seattle-W1
[Observation from check_inventory]
Inventory for Phone Stand:

```

[Output continues below...]

```
Status: In Stock
Units Available: 230
Warehouse: Seattle-W1
[Step 0 - Tool Call #5]
Tool: get_product_details
Input: {'product_name': 'Noise Cancelling Headphones'}
[Step 0 - Tool Call #6]
Tool: get_product_details
Input: {'product_name': 'Laptop Pro 15'}
[Step 0 - Tool Call #7]
Tool: get_product_details
Input: {'product_name': 'Phone Stand'}
[Observation from get_product_details]
Product: Noise Cancelling Headphones
Price: $129.99
Category: Audio
Warranty: 18 months
Description: Over-ear headphones with active noise cancellation and 30hr battery
Rating: 4.8/5.0
[Observation from get_product_details]
Product: Laptop Pro 15
Price: $1299.99
Category: Computers
Warranty: 24 months
Description: 15-inch professional laptop with 16GB RAM and 512GB SSD
Rating: 4.7/5.0
[Observation from get_product_details]
Product: Phone Stand
Price: $24.99
Category: Accessories
Warranty: 6 months
Description: Adjustable aluminum phone and tablet stand
Rating: 4.2/5.0
[Step 1 - Agent Reasoning]
```

[Output continues below...]

```
Here's the summary of your order and warranty coverage:
**Inventory Status:**
- **Noise Cancelling Headphones**: Out of stock (0 units available) - may be challenging to
replace immediately
- **Laptop Pro 15**: In stock (12 units available) - easy to replace if needed
- **Phone Stand**: In stock (230 units available) - easy to replace if needed
**Warranty Coverage:**
- **Noise Cancelling Headphones**: 18 months
- **Laptop Pro 15**: 24 months
- **Phone Stand**: 6 months
**Total combined warranty coverage**: 48 months (4 years)
The out-of-stock headphones may require special order or backorder processing if replacement
is needed. Let me know if you'd like assistance with next steps!
=====
Trace complete. Tool calls made: 7
=====
```

This query pushed the agent through a multi-step process:

1. Look up order 1003 to get the list of items
2. Check inventory for each item (3 separate calls)
3. Get product details for warranty information (3 separate calls)
4. Reason across all the results to identify which items are at risk and calculate warranty coverage

This is the power of ReAct. The agent navigated through potentially 7 tool calls, adapted at each step, and produced a synthesized answer that no single tool call could have provided.

Query 4: Handling Edge Cases

Let us see how the agent handles a tricky situation.

Code

```
run_agent(  
  "I want to reorder everything from order 1004. Is that possible? "  
  "Check if all items are available."  
)
```

Output

=====

QUERY: I want to reorder everything from order 1004. Is that possible? Check if all items are available.

=====

[Step 0 - Tool Call #1]

Tool: lookup_order

Input: {'order_id': '1004'}

[Observation from lookup_order]

Order 1004:

Customer: David Kim

Items: Wireless Mouse, Phone Stand

Status: Cancelled

Order Date: 2024-11-15

Total: \$74.98

Cancellation Reason: Customer requested cancellation

[Step 0 - Tool Call #2]

Tool: check_inventory

Input: {'product_name': 'Wireless Mouse'}

[Step 0 - Tool Call #3]

Tool: check_inventory

Input: {'product_name': 'Phone Stand'}

[Observation from check_inventory]

Inventory for Phone Stand:

Status: In Stock

Units Available: 230

Warehouse: Seattle-W1

[Observation from check_inventory]

Inventory for Wireless Mouse:

Status: In Stock

Units Available: 145

Warehouse: Seattle-W2

[Step 1 - Agent Reasoning]

Both items from your cancelled order (Order 1004) are currently in stock:

- **Wireless Mouse**: 145 units available in Seattle-W2 warehouse

- **Phone Stand**: 230 units available in Seattle-W1 warehouse

[Output continues below...]

```
Would you like me to help you place a new order for these items? I can check current pricing
and ensure availability before proceeding.
```

```
=====
```

```
Trace complete. Tool calls made: 3
```

```
=====
```

This is interesting because order 1004 was cancelled. The agent had to:

1. Look up the cancelled order
2. Recognize it was cancelled and understand the customer wants to re-place it
3. Check inventory for each item
4. Give a recommendation based on availability

The agent handled the edge case naturally because ReAct lets it reason about unexpected situations at each step.

QUIZ 2

Here is a new query I am about to run:

Compare the ratings and prices of the Mechanical Keyboard and the Noise Cancelling Headphones. Which one is the better value for money?

Before I run this, type in chat: How many tool calls do you think the agent will make? Pick a number between 1 and 4.

Code

```
run_agent(
    "Compare the ratings and prices of the Mechanical Keyboard and the "
    "Noise Cancelling Headphones. Which one is the better value for money?"
)
```

Output

```
=====
QUERY: Compare the ratings and prices of the Mechanical Keyboard and the Noise Cancelling
Headphones. Which one is the better value for money?
=====
[Step 0 - Tool Call #1]
Tool: get_product_details
Input: {'product_name': 'Mechanical Keyboard'}
[Step 0 - Tool Call #2]
Tool: get_product_details
Input: {'product_name': 'Noise Cancelling Headphones'}
[Observation from get_product_details]
Product: Mechanical Keyboard
Price: $124.99
Category: Accessories
Warranty: 24 months
Description: RGB mechanical keyboard with Cherry MX Blue switches
Rating: 4.6/5.0
[Observation from get_product_details]
Product: Noise Cancelling Headphones
Price: $129.99
Category: Audio
Warranty: 18 months
Description: Over-ear headphones with active noise cancellation and 30hr battery
Rating: 4.8/5.0
[Step 1 - Agent Reasoning]
The Noise Cancelling Headphones ($129.99, 4.8/5.0) and Mechanical Keyboard ($124.99,
4.6/5.0) are both highly rated, but here's the comparison:
- Price: The keyboard is slightly cheaper ($5 less), while the headphones cost $5 more.
- Rating: The headphones have a higher rating (4.8 vs. 4.6), indicating stronger customer
satisfaction.
Better value for money?
If prioritizing rating and willing to pay a small premium, the headphones offer
slightly better value. If budget is stricter, the keyboard provides a lower price with
still-excellent reviews. Both are strong choices depending on your needs!
=====
Trace complete. Tool calls made: 2
=====
```

The agent needed 2 tool calls, one for each product. It then compared them using pure reasoning (no tool needed for the comparison itself). If you predicted 2, well done.

Notice that the agent did not need to check inventory or look up orders. It correctly identified that only product details were relevant to a value for money comparison. This is the adaptive nature of ReAct. The agent uses only the tools it needs based on the specific question.

Now let us move on to the second major pattern: Reflection.

Reflection: Concept Deep Dive

[TIME: 0:50 to 1:05]

Self-Reflection and Critique Mechanisms

So far, with ReAct, we have seen how agents can gather external information. But what about improving the quality of their own outputs? This is where reflection comes in.

The Core Idea

Reflection is a pattern where an agent evaluates its own output, identifies weaknesses, and generates an improved version. Think of it as a built-in quality assurance loop.

The simplest form is the Judge-Revise pattern:

Analogy: Netflix's Culture of Retrospection

Netflix is famous for its engineering culture of blameless post-mortems. After every significant incident:

1. The team produces an incident report (this is the generation step).
2. A review group critiques the report: Was the root cause correctly identified? Were the mitigation steps sufficient? Are there gaps in monitoring? (This is the judge step.)
3. The team revises the report and the actual systems based on the feedback (this is the revise step).

This cycle is what makes Netflix's systems increasingly reliable over time. The same principle applies to LLM agents. Without reflection, an agent produces its first-draft output and stops. With reflection, the agent can catch its own mistakes, fill in gaps, and produce higher-quality results.

The Judge-Revise Pattern in Detail

There are two common architectures for reflection:

Architecture 1: Single LLM, Two Roles

The same LLM plays both the generator and the judge, but with different system prompts.

This is simpler but the model might be blind to its own systematic errors.

Architecture 2: Two Different LLMs

A separate, possibly stronger model acts as the judge.

This is more robust because the judge has a different perspective. In practice at companies like Google or Microsoft, you often see a stronger model (like GPT-4 or Gemini Pro) judging outputs from a faster, cheaper model.

Today we will implement Architecture 1 using the raw Gemini SDK, with the same model playing both roles through different system instructions.

When Reflection Helps vs. When It Does Not

Reflection is not a magic bullet. Here is when it works well and when it does not.

Key insight: Reflection is most valuable when the generators errors are the kind that become obvious upon review. This is similar to how you often spot bugs in your own code when you come back to review it after a break.

QUIZ 3

Which of the following scenarios would benefit MOST from a reflection pattern? Type A, B, or C in chat.

- A. An agent that needs to translate English to French. The LLM consistently mistranslates idiomatic expressions.
- B. An agent that generates SQL queries. Sometimes it forgets a WHERE clause or uses the wrong JOIN type.
- C. An agent that needs to predict stock prices based on historical data.

Answer: B

* A describes a systematic error. The model keeps making the same idiomatic mistakes, meaning reflection is unlikely to fix it because the model does not know the correct translation.

* B is a classic case where reflection shines. Forgetting a WHERE clause or using the wrong JOIN type are errors that become obvious when you re-read the query. A judge prompt like Check if this query correctly filters the data and uses appropriate joins would catch these.

* C is not a generation quality problem. It is a fundamental capability limitation. No amount of self-reflection will make an LLM good at stock prediction.

Now let us build it.

Reflection: Runnable Demo

[TIME: 1:05 to 1:25]

Building a Judge-Revise Loop for Code Generation

We will build a system where:

1. A Generator writes a Python function based on a specification
2. A Judge reviews the code for correctness, edge cases, and style
3. The Generator revises the code based on the judges feedback
4. We can run this loop multiple times to see iterative improvement

We are using the raw Gemini SDK here. No LangChain, no chains, no abstractions. Just direct function calls to the model with different system instructions. This makes the reflection pattern completely transparent.

Code

```
from langchain_groq import ChatGroq
from langchain_core.messages import SystemMessage, HumanMessage
import os
llm=ChatGroq(model="qwen/qwen3-32b", temperature=0, api_key=os.environ["GROQ_API_KEY"])
def _ask(sys,u): return
llm.invoke([SystemMessage(content=sys), HumanMessage(content=u)]).content
def generate_code(spec): return _ask("You are an expert Python developer. Return only Python
code.", spec)
def judge_code(code): return _ask("You are a senior reviewer. Reply LGTM if perfect.", code)
def revise_code(spec, previous_code, feedback): return _ask("You are an expert Python
developer.", f"Specification:\n{spec}\n\nPrevious
Code:\n{previous_code}\n\nFeedback:\n{feedback}")
```

All three functions are working. Notice how simple each one is. There is no framework magic here. Each function:

1. Creates a model instance with a specific
2. Sends a prompt with
3. Returns the text response

This is the raw building block of the Judge-Revise pattern. Now let us build the loop.

Code

```

def reflect_and_revise(spec: str, max_iterations: int = 3, verbose: bool = True) -> dict:
    """
    Run the Judge-Revise reflection loop.
    Args:
    spec: The code specification/requirements
    max_iterations: Maximum number of revision cycles
    verbose: Whether to print each step
    Returns:
    Dictionary with the history of generations, critiques, and the final code
    """
    history = {"iterations": []}
    # Step 1: Initial Generation
    if verbose:
        print("=" * 70)
        print("INITIAL GENERATION")
        print("=" * 70)
    current_code = generate_code(spec)
    if verbose:
        print(current_code)
    for i in range(max_iterations):
        if verbose:
            print(f"\n{'=' * 70}")
            print(f"JUDGE REVIEW - ITERATION {i + 1}")
            print(f"{'=' * 70}")
        # Step 2: Judge reviews the code
        critique = judge_code(current_code)
        if verbose:
            print(f"\nFEEDBACK:\n{critique}")
        iteration_record = {
            "iteration": i + 1,
            "code": current_code,
            "critique": critique
        }
        history["iterations"].append(iteration_record)
        # Check if the judge approves
        if "LGTM" in critique.strip().upper():
            if verbose:
                print(f"\nJudge approved the code at iteration {i + 1}!")
            history["final_code"] = current_code
            history["total_iterations"] = i + 1
            history["approved"] = True
            return history
        # Step 3: Revise based on feedback
        if verbose:
            print(f"\n{'=' * 70}")
            print(f"REVISION - ITERATION {i + 1}")
            print(f"{'=' * 70}")
        revised_code = revise_code(spec, current_code, critique)
        if verbose:
            print(f"\nREVISED CODE:\n{revised_code}")
        current_code = revised_code
        history["final_code"] = current_code
        history["total_iterations"] = max_iterations
        history["approved"] = False
    return history
    print("Reflection loop is ready.")

```


Output

```
Reflection loop is ready.
```

Let us test this with a specification that is complex enough to trigger meaningful critique on the first attempt.

Code

```
spec_1 = """
Write a Python function called 'merge_sorted_lists' that takes two sorted lists
of integers and returns a single sorted list containing all elements from both lists.
Requirements:
- Handle empty lists
- Handle lists of different lengths
- Handle duplicate values
- Do NOT use Python's built-in sort() or sorted()
- Time complexity should be O(n + m) where n and m are the lengths of the input lists
"""
result_1 = reflect_and_revise(spec_1, max_iterations=3)
```

Output

```
=====
INITIAL GENERATION
=====
```

<think>

Okay, I need to write a Python function called `merge_sorted_lists` that takes two sorted lists and returns a merged sorted list. Let me think about how to approach this.

The problem says not to use the built-in `sort` or `sorted` functions. Oh right, this is probably meant to implement the merge step from the merge sort algorithm. Because merge sort's merge step does exactly that—combines two sorted lists into one in $O(n + m)$ time.

So the plan is to use two pointers, one for each list. Compare the current elements of both lists, take the smaller one, add it to the result, and move the pointer. Repeat until all elements are processed.

Let me outline the steps:

- Initialize an empty list to hold the merged result. Let's call it `merged_list`.
- Initialize two pointers, `i` and `j`, starting at 0 for `list1` and `list2` respectively.
- Loop while `i` is less than `len(list1)` and `j` is less than `len(list2)`. Compare `list1[i]` and `list2[j]`. If `list1[i]` is smaller, append it to `merged_list` and increment `i`. Else, append `list2[j]` and increment `j`.
- Once one of the lists is exhausted, add the remaining elements from the other list to `merged_list`. For example, if there are elements left in `list1` (`i` hasn't reached the end), add all remaining elements from `list1` starting at `i`. Similarly for `list2`.

Wait, but what about duplicates? Oh, since the original lists are already sorted, duplicates are handled naturally. For example, if `list1` has `[1, 2, 2]` and `list2` has `[2, 3]`, merging them by comparing each step would result in `[1, 2, 2, 2, 3]`. Which is correct.

Now, handling empty lists. Suppose one of the lists is empty. For example, `list1` is empty, return `list2`. Similarly if `list2` is empty, return `list1`. The code should handle that because in the initial loop, if one is empty, the loop doesn't run, and then the remaining elements (all from the other list) are added.

Let me test this logic with some examples.

Test case 1: `list1 = [1, 3, 5]`, `list2 = [2, 4, 6]`. Merged should be `[1,2,3,4,5,6]`. Let's see: Compare 1 and 2 ? take 1. `i` becomes 1. Next compare 3 and 2 ? take 2. `j` becomes 1. Then compare 3 and 4 ? take 3. `i` becomes 2. Compare 5 and 4 ? take 4. `j` becomes 2. Compare 5 and 6 ? take 5. `i` becomes 3. Now `list1` is exhausted. Add remaining elements from `list2` (6). Result is correct.

Another test case where one list is empty. Suppose `list1` is `[]`, `list2` is `[1,2]`. The merged list should be `[1,2]`. The code should handle that.

Another case where there are duplicates: `list1 = [1, 2, 2]`, `list2 = [2, 3]`. The merged list should be `[1,2,2,2,3]`. The code would compare and add each step.

Now, coding this in Python.

The function will take `list1` and `list2` as parameters.

```
Initialize i = 0, j = 0. merged_list = [].
Loop while i < len(list1) and j < len(list2):
if list1[i] <= list2[j], append to merged_list and increment i. else append list2[j] and
increment j.
After the loop, there might be elements left in list1 or list2. So, we can check: if i <
len(list1), add all elements from i onwards. Similarly for j.
```

[Output continues below...]

Wait, but adding the remaining elements can be done with list slicing. For example, after the loop, `merged_list += list1[i:]` or something similar. Because once one of the lists is exhausted, the remaining elements in the other list are already sorted and can be added as is. Yes. Because if one list is longer than the other, this handles it. For example, if `list1` is longer, after `list2` is exhausted, add the remaining elements of `list1`.

So code steps:

```
def merge_sorted_lists(list1, list2):
    merged = []
    i = j = 0
    while i < len(list1) and j < len(list2):
        if list1[i] <= list2[j]:
            merged.append(list1[i])
            i += 1
        else:
            merged.append(list2[j])
            j += 1
    # Now add the remaining elements
    # If there are remaining elements in list1
    while i < len(list1):
        merged.append(list1[i])
        i += 1
    # Or if there are remaining elements in list2
    while j < len(list2):
        merged.append(list2[j])
        j += 1
    return merged
```

Wait, but what about when one of the lists is empty? Like `list1` is empty. Then the first while loop doesn't run. Then the code adds the remaining elements from `list2`, which is the entire `list2`. Which works.

Testing with empty lists. Let's see:

Case where `list1` is `[1,3,5]`, `list2` is empty. The merged list is `[1,3,5]`. Correct.

Another case where both are empty, returns empty.

Yes.

Time complexity: Each element is processed once. So $O(n + m)$, which meets the requirement.

So this function should satisfy all the requirements. Let me check the requirements again:

[Output continues below...]

- Handle empty lists: yes.
- Different lengths: yes, because the code adds remaining elements.
- Duplicates: handled correctly since comparing each step.
- No use of sort or sorted: correct.

Yes. So this should work.

Now, code this function.

</think>

```
```python
def merge_sorted_lists(list1, list2):
 merged = []
 i = j = 0
 while i < len(list1) and j < len(list2):
 if list1[i] <= list2[j]:
 merged.append(list1[i])
 i += 1
 else:
 merged.append(list2[j])
 j += 1
 while i < len(list1):
 merged.append(list1[i])
 i += 1
 while j < len(list2):
 merged.append(list2[j])
 j += 1
 return merged
```
```

=====

JUDGE REVIEW - ITERATION 1

=====

FEEDBACK:

<think>

Okay, I need to review the provided Python function `merge\_sorted\_lists`. Let me start by understanding what it's supposed to do. The function takes two sorted lists and merges them into a single sorted list without using built-in sort functions.

Looking at the code, it uses two pointers `i` and `j` to iterate through `list1` and `list2`, respectively. The main loop compares elements at these pointers and appends the smaller one to the `merged` list. After one of the lists is exhausted, the remaining elements from the other list are added to `merged`.

First, I'll check if the code handles edge cases correctly. What if one of the lists is empty? For example, if `list1` is empty, the first while loop doesn't run, and the code appends all elements from `list2` in the second loop. That works. Similarly, if `list2` is empty, the first loop doesn't run, and the remaining elements from `list1` are added. So empty lists are handled properly.

[Output continues below...]

Next, what about lists of different lengths? Suppose `list1` is longer than `list2`. After the first loop, `j` might have reached the end of `list2`, and the remaining elements in `list1` are added. That's correct. The same applies if `list2` is longer.

Now, checking for duplicates. If both lists have the same element, like `list1 = [2, 2]` and `list2 = [2, 3]`, the merged list should be `[2, 2, 2, 3]`. The code compares `list1[i] <= list2[j]`, so it appends the first `2` from `list1`, then the next `2` from `list1`, and so on. That's correct.

I'll test a sample case. Let's take `list1 = [1, 3, 5]` and `list2 = [2, 4, 6]`. The expected merged list is `[1, 2, 3, 4, 5, 6]`. The code's first loop compares 1 vs 2 (append 1), then 3 vs 2 (append 2), then 3 vs 4 (append 3), then 5 vs 4 (append 4), then 5 vs 6 (append 5), and finally append 6. That works.

Another test case: `list1 = [1, 2, 2]` and `list2 = [2, 3]`. The merged list should be `[1, 2, 2, 2, 3]`. The code appends 1, then compares 2 vs 2 (appends both in order), then appends remaining 2 from `list1`, then 3 from `list2`. Correct.

What about when one list is completely exhausted early? For example, `list1 = [1, 3]` and `list2 = [2]`. The first loop appends 1, then compares 3 vs 2 (appends 2), then appends 3. The merged list is `[1, 2, 3]`. Correct.

The code uses two additional while loops after the main loop to append remaining elements.

This ensures that any leftover elements from either list are added to the merged list. This is necessary because once one list is exhausted, the other might still have elements that are already in order.

I don't see any use of Python's built-in `sort` or `sorted` functions, which aligns with the problem constraints. The time complexity is $O(n + m)$, where n and m are the lengths of the two lists, which is optimal for this task.

Potential issues? Let me think. What if the input lists are not sorted? The function assumes they are sorted, which is part of the problem's constraints. Since the problem states that the

inputs are sorted, the function doesn't need to handle unsorted inputs. So that's acceptable. Another thing to check: the code uses ``<=`` in the comparison. This ensures that if elements are equal, the element from `list1` is added first. This maintains stability if needed, but the problem doesn't specify stability requirements. However, since the problem says the input lists are sorted, this approach is correct.

Overall, the code seems to handle all required cases correctly. The logic is sound, and the implementation is efficient. I don't see any bugs or issues that need fixing.

`</think>`

LGTM

Judge approved the code at iteration 1!

Let us examine how the code improved across iterations.

Code

```
print(f"Total review iterations: {result_1['total_iterations']}")
print(f"Judge approved: {result_1['approved']}")
for iteration in result_1["iterations"]:
    print(f"\n{'=' * 50}")
    print(f"Iteration {iteration['iteration']} - Critique Summary:")
    print(f"{'=' * 50}")
    # Show first 400 characters of the critique
    critique_preview = iteration["critique"][:400]
    print(critique_preview)
    if len(iteration["critique"]) > 400:
        print("...")
        print(f"\n{'=' * 50}")
    print("FINAL VERSION:")
    print(f"{'=' * 50}")
    print(result_1["final_code"])
```

Output

```

Total review iterations: 1
Judge approved: True
=====
Iteration 1 - Critique Summary:
=====
<think>
Okay, I need to review the provided Python function `merge_sorted_lists`. Let me start by
understanding what it's supposed to do. The function takes two sorted lists and merges them
into a single sorted list without using built-in sort functions.
Looking at the code, it uses two pointers `i` and `j` to iterate through `list1` and `list2`,
respectively. The main loop compares elements at
...
=====
FINAL VERSION:
=====
<think>
Okay, I need to write a Python function called merge_sorted_lists that takes two sorted lists
and returns a merged sorted list. Let me think about how to approach this.
The problem says not to use the built-in sort or sorted functions. Oh right, this is probably
meant to implement the merge step from the merge sort algorithm. Because merge sort's merge
step does exactly that-combines two sorted lists into one in  $O(n + m)$  time.
So the plan is to use two pointers, one for each list. Compare the current elements of both
lists, take the smaller one, add it to the result, and move the pointer. Repeat until all
elements are processed.
Let me outline the steps:
Initialize an empty list to hold the merged result. Let's call it merged_list.
Initialize two pointers, i and j, starting at 0 for list1 and list2 respectively.
Loop while i is less than len(list1) and j is less than len(list2). Compare list1[i] and
list2[j]. If list1[i] is smaller, append it to merged_list and increment i. Else, append
list2[j] and increment j.
Once one of the lists is exhausted, add the remaining elements from the other list to
merged_list. For example, if there are elements left in list1 (i hasn't reached the end), add
all remaining elements from list1 starting at i. Similarly for list2.
Wait, but what about duplicates? Oh, since the original lists are already sorted, duplicates
are handled naturally. For example, if list1 has [1, 2, 2] and list2 has [2, 3], merging them
by comparing each step would result in [1, 2, 2, 2, 3]. Which is correct.
Now, handling empty lists. Suppose one of the lists is empty. For example, list1 is empty,
return list2. Similarly if list2 is empty, return list1. The code should handle that because
in the initial loop, if one is empty, the loop doesn't run, and then the remaining elements
(all from the other list) are added.
Let me test this logic with some examples.
Test case 1: list1 = [1, 3, 5], list2 = [2, 4, 6]. Merged should be [1,2,3,4,5,6]. Let's see:
Compare 1 and 2 ? take 1. i becomes 1. Next compare 3 and 2 ? take 2. j becomes 1. Then
compare 3 and 4 ? take 3. i becomes 2. Compare 5 and 4 ? take 4. j becomes 2. Compare 5 and 6
? take 5. i becomes 3. Now list1 is exhausted. Add remaining elements from list2 (6). Result
is correct.
Another test case where one list is empty. Suppose list1 is [], list2 is [1,2]. The merged

```


list should be [1,2]. The code should handle that.

Another case where there are duplicates: list1 = [1, 2, 2], list2 = [2, 3]. The merged list should be [1,2,2,2,3]. The code would compare and add each step.

[Output continues below...]

Now, coding this in Python.

The function will take list1 and list2 as parameters.

Initialize `i = 0`, `j = 0`. `merged_list = []`.

Loop while `i < len(list1)` and `j < len(list2)`:

if `list1[i] <= list2[j]`, append to `merged_list` and increment `i`. else append `list2[j]` and increment `j`.

After the loop, there might be elements left in list1 or list2. So, we can check: if `i < len(list1)`, add all elements from `i` onwards. Similarly for `j`.

Wait, but adding the remaining elements can be done with list slicing. For example, after the loop, `merged_list += list1[i:]` or something similar. Because once one of the lists is exhausted, the remaining elements in the other list are already sorted and can be added as is. Yes. Because if one list is longer than the other, this handles it. For example, if list1 is longer, after list2 is exhausted, add the remaining elements of list1.

So code steps:

```
def merge_sorted_lists(list1, list2):
```

```
    merged = []
```

```
    i = j = 0
```

```
    while i < len(list1) and j < len(list2):
```

```
        if list1[i] <= list2[j]:
```

```
            merged.append(list1[i])
```

```
            i += 1
```

```
        else:
```

```
            merged.append(list2[j])
```

```
            j += 1
```

```
    # Now add the remaining elements
```

```
    # If there are remaining elements in list1
```

```
    while i < len(list1):
```

```
        merged.append(list1[i])
```

```
        i += 1
```

```
    # Or if there are remaining elements in list2
```

```
    while j < len(list2):
```

```
        merged.append(list2[j])
```

```
        j += 1
```

```
    return merged
```

Wait, but what about when one of the lists is empty? Like list1 is empty. Then the first while loop doesn't run. Then the code adds the remaining elements from list2, which is the entire list2. Which works.

[Output continues below...]

Testing with empty lists. Let's see:

Case where list1 is [1,3,5], list2 is empty. The merged list is [1,3,5]. Correct.

Another case where both are empty, returns empty.

Yes.

Time complexity: Each element is processed once. So $O(n + m)$, which meets the requirement.

So this function should satisfy all the requirements. Let me check the requirements again:

- Handle empty lists: yes.
- Different lengths: yes, because the code adds remaining elements.
- Duplicates: handled correctly since comparing each step.
- No use of sort or sorted: correct.

Yes. So this should work.

Now, code this function.

</think>

```
```python
def merge_sorted_lists(list1, list2):
 merged = []
 i = j = 0
 while i < len(list1) and j < len(list2):
 if list1[i] <= list2[j]:
 merged.append(list1[i])
 i += 1
 else:
 merged.append(list2[j])
 j += 1
 while i < len(list1):
 merged.append(list1[i])
 i += 1
 while j < len(list2):
 merged.append(list2[j])
 j += 1
 return merged
```
```

Let us Verify the Final Code Actually Works

One of the strengths of code generation with reflection is that we can actually test the output. Let us execute the final code and run some test cases.

Code

```
import re
# Extract only the Python code from the final_code string
code_match = re.search(r"```python\n(?:.*?)```", result_1["final_code"], re.DOTALL)
if code_match:
    executable_code = code_match.group(1)
else:
    executable_code = result_1["final_code"] # Fallback if regex fails, though it might still
    error
# Execute the final generated code
exec(executable_code)
# Run test cases
print("Test 1 - Normal case:")
print(f" merge_sorted_lists([1, 3, 5], [2, 4, 6]) = {merge_sorted_lists([1, 3, 5], [2, 4, 6])}")
print("\nTest 2 - Empty lists:")
print(f" merge_sorted_lists([], [1, 2, 3]) = {merge_sorted_lists([], [1, 2, 3])}")
print(f" merge_sorted_lists([1, 2], []) = {merge_sorted_lists([1, 2], [])}")
print(f" merge_sorted_lists([], []) = {merge_sorted_lists([], [])}")
print("\nTest 3 - Duplicates:")
print(f" merge_sorted_lists([1, 2, 2], [2, 3, 3]) = {merge_sorted_lists([1, 2, 2], [2, 3, 3])}")
print("\nTest 4 - Different lengths:")
print(f" merge_sorted_lists([1], [2, 3, 4, 5]) = {merge_sorted_lists([1], [2, 3, 4, 5])}")
```

Output

```
Test 1 - Normal case:
merge_sorted_lists([1, 3, 5], [2, 4, 6]) = [1, 2, 3, 4, 5, 6]
Test 2 - Empty lists:
merge_sorted_lists([], [1, 2, 3]) = [1, 2, 3]
merge_sorted_lists([1, 2], []) = [1, 2]
merge_sorted_lists([], []) = []
Test 3 - Duplicates:
merge_sorted_lists([1, 2, 2], [2, 3, 3]) = [1, 2, 2, 2, 3, 3]
Test 4 - Different lengths:
merge_sorted_lists([1], [2, 3, 4, 5]) = [1, 2, 3, 4, 5]
```

The code works correctly across all test cases. This is a tangible demonstration of why reflection matters. The initial generation might have missed edge cases, but the judge caught them and the revision addressed them.

QUIZ 4

Let us run another reflection cycle with a different specification. Before I run it, read the spec below and predict in chat: What is the most likely issue the Judge will flag on the first attempt?

Type your prediction in chat.

Code

```
spec_2 = """
Write a Python function called 'flatten_dict' that takes a nested dictionary
and returns a flattened dictionary where nested keys are joined with a dot separator.
Example:
Input: {"a": 1, "b": {"c": 2, "d": {"e": 3}}}
Output: {"a": 1, "b.c": 2, "b.d.e": 3}
Requirements:
- Handle arbitrary nesting depth
- Handle empty nested dictionaries
- Handle cases where values are lists (keep lists as-is, do not flatten them)
"""
result_2 = reflect_and_revise(spec_2, max_iterations=3)
```

Output

```
=====
INITIAL GENERATION
=====
<think>
Okay, I need to write a Python function called flatten_dict that takes a nested dictionary and
returns a flattened version. The keys in the output should be joined with dots. Let me think
about how to approach this.
First, the example given is {"a": 1, "b": {"c": 2, "d": {"e": 3}}} which becomes {"a": 1,
"b.c": 2, "b.d.e": 3}. So for each nested level, we prepend the parent key with a dot.
The function needs to handle arbitrary nesting. So recursion might be a good approach here.
For each key in the input dictionary, if the value is another dictionary, we recursively
flatten it. But how to accumulate the keys?
Let me think of a helper function that takes the current dictionary, a parent key, and
accumulates into the result. For example, for each key-value pair in the current dict, if the
value is a dict, we call the helper again with the parent key being the current key appended
to the existing parent. Otherwise, we add the combined key to the result.
Wait, but how to start? The initial call would have an empty parent key. Let me outline steps:
Initialize a result dictionary. For each key, value in the input dict:
- If the value is a dict, then recursively process that value, passing the current key as the
parent. Then merge the returned dict into the result.
- Else, add the key (with parent prefix) to the result.
But how to handle the parent key during recursion. Maybe a helper function that takes the
current dictionary and the parent key.
So the main function would call this helper, starting with an empty parent key.
Let me sketch this:
def flatten_dict(d):
def _flatten(current, parent_key, result):
for k, v in current.items():
new_key = parent_key + '.' + k if parent_key else k
if isinstance(v, dict):
_flatten(v, new_key, result)
else:
result[new_key] = v
result = {}
_flatten(d, '', result)
return result
Wait, but in the example, for the input {"a": 1, "b": {"c": 2, ...}}, the first level key 'a'
has parent_key empty. So new_key is 'a'. For 'b', we check if it's a dict. So we call _flatten
on the value of 'b', with parent_key 'b'. Then, in that call, for key 'c', new_key is 'b.c',
and so on.
That seems right. Let me test this logic with the example.
Original input: {"a": 1, "b": {"c": 2, "d": {"e": 3}}}
Processing 'a' with parent_key empty: new_key is 'a', value is 1, so result['a'] = 1.
```

[Output continues below...]

Processing 'b' with parent_key empty: new_key is 'b'. Since value is a dict, call _flatten on that dict with parent_key 'b'.

Inside that, for key 'c', new_key is 'b.c', value 2. Added to result.

Then key 'd', which is a dict. So call _flatten with parent_key 'b.d'. Inside that, key 'e' gives new_key 'b.d.e', value 3. So added to result.

Yes, that gives the correct output.

But wait, what about if a key is added with a parent key that's part of another key? For example, if there's a key 'a.b' and also a key 'a' with a nested 'b'? But in the problem statement, the input is a nested dict, so that scenario would not happen naturally. But maybe in some edge cases. But the function is supposed to handle that? Well, in the output, the keys are just strings, and if two different paths result in the same key, the later one would overwrite. But that's up to the user to handle.

Now, what about empty nested dictionaries? For example, if a key has an empty dict as value. Let's say input is {'a': {}}. The function would process that. Let me see: for each key in the empty dict, there are none. So the _flatten function would loop zero times, and the result would have nothing. So the output would be empty? But the original key 'a' is not present. Wait, but according to the requirements, handle empty nested dictionaries. So perhaps, if a dict is empty, we should not include it. But since there are no key-value pairs, they don't contribute to the result. So that's correct.

Another case: if a value is a list. According to the requirements, lists should be kept as-is. So if a key's value is a list, we just add it to the result with the current key. For example, if input is {'a': [1,2,3]}, then output is {'a': [1,2,3]}.

But in the code above, since it checks if v is a dict, then it's not a list, so it would add the new_key with the list as value. So that's handled.

Wait, but what if a value is a list containing dicts? Well, according to the requirements, the function should not flatten lists. So the code treats the list as a non-dict value, so the entire list is added as-is. For example, input {'a': [{'b': 1}]}, the output would be {'a': [{'b': 1}]}.

But what if the code encounters a list? Let me think. Suppose input is {'a': {'b': {'c': 2}, 'd': [1, 2, 3]}}, then the code would process 'b' as a dict, leading to 'a.b.c' : 2, and 'd' is a list, so added as 'a.d': [1,2,3].

Yes, that's correct.

Now, testing the example. Let me code this and see.

But what about the initial call? Let me think about when the input is empty. But the problem says it's a dictionary, but perhaps empty. But the function is supposed to handle that. For example, input {} returns {}.

Another edge case: a single-level dict. Like {'a': 1, 'b': 2} becomes {'a':1, 'b':2}.

Another edge case: nested dicts. Let me think of a more complex example.

But let me code the function. The helper function is inside flatten_dict. Now, the code as I sketched.

But wait, the initial parent_key is empty string. Let's see for the first level key. For example, in the example, the key 'b' becomes parent_key 'b' for the next level. Suppose in the next level, the key is 'c', then new_key is 'b.c'.

But what if the original dictionary has a key that is a string with dots in it? Like, if input is {'a.b': 1}. Then the output would have key 'a.b' which is correct. Because in the original, it's a single key. So the code would generate that.

Now, what about if a value is a dict that's empty? For example, input {'a': {}}. Then during

processing, the code loops over the items in the dict, but there are none. So nothing is added. So the output is empty? But the original input has a key 'a' with value empty dict. But according to the problem statement, empty nested dictionaries should be handled. But what is expected? Since the value is empty, there's nothing to flatten. So the output should not have the key 'a' because the value is empty. So that's correct. For example, input {'a': {}} would result in {}.

But what if the user wants to keep empty dicts? Well, the problem says to handle them. But since there's no value to put, perhaps it's just ignored. So the code as written handles that correctly.

Now, testing the code.

Let me try the example input:

Input: {"a": 1, "b": {"c": 2, "d": {"e": 3}}}

[Output continues below...]

```
The code would process:
Start with parent_key empty.
For key 'a', new_key is 'a', value 1 added to result.
For key 'b', value is a dict. So call _flatten on that dict with parent_key 'b'.
Inside that, for key 'c', new_key is 'b.c', value 2 added.
For key 'd', value is a dict. Call _flatten with parent_key 'b.d'.
Inside that, key 'e', new_key is 'b.d.e', value 3 added.
So the output is correct.
Another test case: input {'a': {'b': {'c': {}}}}. The innermost dict is empty. So during
processing, when processing 'c' which has an empty dict, nothing is added. So the output would
be empty? Or would the code add the key 'a.b.c' with value {}? Let's see. The value is a dict
(empty), so the code would call _flatten on it. Then, in the loop over items, there are no
items. So nothing is added. Therefore, the output would be empty. But the original input has
two nested empty dicts. So the output is empty. Which is correct according to the problem's
requirements? Well, the problem says to handle empty nested dictionaries. But since there's no
value to output, they are just omitted. So that's acceptable.
Another test case: input {'a': [1, 2,
=====
JUDGE REVIEW - ITERATION 1
=====
FEEDBACK:
<think>
Okay, I need to write a Python function called `flatten_dict` that takes a nested dictionary
and returns a flattened version where the keys are joined with dots. Let me think about how to
approach this.
First, the example given is `{"a": 1, "b": {"c": 2, "d": {"e": 3}}}` which becomes `{"a": 1,
"b.c": 2, "b.d.e": 3}`. So for each nested level, we prepend the parent key with a dot.
The function needs to handle arbitrary nesting. So recursion might be a good approach here.
For each key in the input dictionary, if the value is another dictionary, we recursively
flatten it. But how to accumulate the keys?
Let me think of a helper function that takes the current dictionary, a parent key, and
accumulates into the result. For example, for each key-value pair in the current dict, if the
value is a dict, we call the helper again with the parent key being the current key appended
to the existing parent. Otherwise, we add the combined key to the result.
Wait, but how to start? The initial call would have an empty parent key. Let me outline steps:
Initialize a result dictionary. For each key, value in the input dict:
- If the value is a dict, then recursively process that value, passing the current key as the
parent. Then merge the returned dict into the result.
- Else, add the key (with parent prefix) to the result.
But how to handle the parent key during recursion. Maybe a helper function that takes the
current dictionary and the parent key.
So the main function would call this helper, starting with an empty parent key.
```

[Output continues below...]

Let me sketch this:

```
```python
def flatten_dict(d):
def _flatten(current, parent_key, result):
for k, v in current.items():
new_key = parent_key + '.' + k if parent_key else k
if isinstance(v, dict):
_flatten(v, new_key, result)
else:
result[new_key] = v
result = {}
_flatten(d, '', result)
return result
```
```

Wait, but in the example, for the input `{ "a": 1, "b": { "c": 2, "d": { "e": 3 } } }`:

Processing 'a' with parent_key empty: new_key is 'a', value is 1, so result['a'] = 1.

Processing 'b' with parent_key empty: new_key is 'b'. Since value is a dict, call `_flatten` on that dict with parent_key 'b'.

Inside that, for key 'c', new_key is 'b.c', value 2. Added to result.

Then key 'd', which is a dict. So call `_flatten` with parent_key 'b.d'. Inside that, key 'e' gives new_key 'b.d.e', value 3. So added to result.

Yes, that gives the correct output.

But wait, what about if a key is added with a parent key that's part of another key? For example, if there's a key 'a.b' and also a key 'a' with a nested 'b'? But in the problem statement, the input is a nested dict, so that scenario would not happen naturally. But the function is supposed to handle that? Well, in the output, the keys are just strings, and if two different paths result in the same key, the later one would overwrite. But that's up to the user to handle.

Now, what about empty nested dictionaries? For example, if a key has an empty dict as value. Let's say input is `{ 'a': {} }`. The function would process that. Let me see: for each key in the empty dict, there are none. So the `_flatten` function would loop zero times, and the result would have nothing. So the output would be empty? But according to the requirements, handle empty nested dictionaries. So perhaps, if a dict is empty, we should not include it. But since there are no key-value pairs, they don't contribute to the result. So that's correct.

Another case: if a value is a list. According to the requirements, lists should be kept as-is. So if a key's value is a list, we just add it to the result with the current key. For example, if input is `{ 'a': [1,2,3] }`, then output is `{ 'a': [1,2,3] }`.

But in the code above, since it checks if `v` is a dict, then it's not a list, so it would add the new_key with the list as value. So that's handled.

Wait, but what if a value is a list containing dicts? Well, according to the requirements, the function should not flatten lists. So the code treats the list as a non-dict value, so the entire list is added as-is. For example, input `{ 'a': [{ 'b': 1 }] }` would result in `{ 'a': [{ 'b': 1 }] }`.

Yes, that's correct.

Now, testing the example. Let me code this and see.

But what about the initial call? Let me think about when the input is empty. But the problem

says it's a dictionary, but perhaps empty. The function is supposed to handle that. For example, input ``{}`` returns ``{}``.

[Output continues below...]

Another edge case: a single-level dict. Like `{'a': 1, 'b': 2}` becomes `{'a':1, 'b':2}`.

Another edge case: nested dicts. Let me think of a more complex example.

But let me code the function. The helper function is inside `_flatten_dict`. Now, the code as I sketched.

But wait, the initial `parent_key` is an empty string. Let's see for the first level key. For example, in the example, the key 'b' becomes `parent_key 'b'` for the next level. Suppose in the next level, the key is 'c', then `new_key` is 'b.c'.

But what if the original dictionary has a key that is a string with dots in it? Like, if input is `{'a.b': 1}`. Then the output would have key 'a.b' which is correct. Because in the original, it's a single key. So the code would generate that.

Now, what if a value is a dict that's empty? For example, input `{'a': {}}`. Then during processing, the code loops over the items in the dict, but there are none. So nothing is added. So the output is empty? But the original input has a key 'a' with value empty dict. But according to the problem statement, empty nested dictionaries should be handled. But since there's nothing to flatten, the output should not have the key 'a' because the value is empty. So that's correct. For example, input `{'a': {}}` would result in `{}`.

But what if the user wants to keep empty dicts? Well, the problem says to handle them. But since there's no value to put, perhaps it's just ignored. So the code as written handles that correctly.

Now, testing the code.

Let me try the example input:

Input: `{ "a": 1, "b": { "c": 2, "d": { "e": 3 } } }`

The code would process:

Start with `parent_key` empty.

For key 'a', `new_key` is 'a', value 1 added to result.

For key 'b', value is a dict. So call `_flatten` on that dict with `parent_key 'b'`.

Inside that, for key 'c', `new_key` is 'b.c', value 2 added.

For key 'd', value is a dict. Call `_flatten` with `parent_key 'b.d'`.

Inside that, key 'e', `new_key` is 'b.d.e', value 3 added.

So the output is correct.

Another test case: input `{'a': {'b': {'c': {}}}}`. The innermost dict is empty. So during processing, when processing 'c' which has an empty dict, nothing is added. So the output would be empty? Or would the code add the key 'a.b.c' with value `{}`? Let's see. The value is a dict (empty), so the code would call `_flatten` on it. Then, in the loop over items, there are no items. So nothing is added. Therefore, the output would be empty. But the original input has two nested empty dicts. So the output is empty. Which is correct according to the problem's requirements? Well, the problem says to handle empty nested dictionaries. But since there's no value to output, they are just omitted. So that's acceptable.

Another test case: input `{'a': [1, 2, 3]}`. The code would add 'a' with value [1,2,3], which is correct.

Another test case: input `{'a': {'b': {'c': 4}, 'd': [5,6,7]}}`. The output should be `{'a.b.c': 4, 'a.d': [5,6,7]}`.

Yes, the code would handle that.

Now, what about if a value is a list containing a dict? For example, input `{'a': [{'b': 1}]}`. The code would add 'a' with value `[{'b': 1}]`, which is correct.

So the code seems to handle all the cases. Now, let me check for any possible issues. What if the input is not a dictionary? Well, the problem states that the input is a dictionary, so we can assume

```
=====
REVISION - ITERATION 1
=====
```

REVISED CODE:

<think>

Okay, I need to write a Python function called `flatten\_dict` that takes a nested dictionary and returns a flattened version where the keys are joined with dots. Let me think about how to approach this.

First, the example given is `{ "a": 1, "b": { "c": 2, "d": { "e": 3 } } }` which becomes `{ "a": 1, "b.c": 2, "b.d.e": 3 }`. So for each nested level, we prepend the parent key with a dot.

The function needs to handle arbitrary nesting. So recursion might be a good approach here.

For each key in the input dictionary, if the value is another dictionary, we recursively flatten it. But how to accumulate the keys?

Let me think of a helper function that takes the current dictionary, a parent key, and accumulates into the result. For example, for each key-value pair in the current dict, if the value is a dict, we call the helper again with the parent key being the current key appended to the existing parent. Otherwise, we add the combined key to the result.

Wait, but how to start? The initial call would have an empty parent key. Let me outline steps: Initialize a result dictionary. For each key, value in the input dict:

- If the value is a dict, then recursively process that value, passing the current key as the parent. Then merge the returned dict into the result.
- Else, add the key (with parent prefix) to the result.

But how to handle the parent key during recursion. Maybe a helper function that takes the current dictionary and the parent key.

So the main function would call this helper, starting with an empty parent key.

Let me sketch this:

```
```python
def flatten_dict(d):
 def _flatten(current, parent_key, result):
 for k, v in current.items():
 new_key = parent_key + '.' + k if parent_key else k
 if isinstance(v, dict):
 _flatten(v, new_key, result)
 else:
 result[new_key] = v
 result = {}
 _flatten(d, '', result)
 return result
```
```

[Output continues below...]

Wait, but in the example, for the input ``{"a": 1, "b": {"c": 2, "d": {"e": 3}}}``:

Processing 'a' with parent_key empty: new_key is 'a', value is 1, so `result['a'] = 1`.

Processing 'b' with parent_key empty: new_key is 'b'. Since value is a dict, call `_flatten` on that dict with parent_key 'b'.

Inside that, for key 'c', new_key is 'b.c', value 2. Added to result.

Then key 'd', which is a dict. So call `_flatten` with parent_key 'b.d'. Inside that, key 'e' gives new_key 'b.d.e', value 3. So added to result.

Yes, that gives the correct output.

But wait, what about if a key is added with a parent key that's part of another key? For example, if there's a key 'a.b' and also a key 'a' with a nested 'b'? But in the problem statement, the input is a nested dict, so that scenario would not happen naturally. But the function is supposed to handle that? Well, in the output, the keys are just strings, and if two different paths result in the same key, the later one would overwrite. But that's up to the user to handle.

Now, what about empty nested dictionaries? For example, if a key has an empty dict as value. Let's say input is ``{'a': {}}``. The function would process that. Let me see: for each key in the empty dict, there are none. So the `_flatten` function would loop zero times, and the result would have nothing. So the output would be empty? But according to the requirements, handle empty nested dictionaries. So perhaps, if a dict is empty, we should not include it. But since there are no key-value pairs, they don't contribute to the result. So that's correct.

Another case: if a value is a list. According to the requirements, lists should be kept as-is. So if a key's value is a list, we just add it to the result with the current key. For example, if input is ``{'a': [1,2,3]}``, then output is ``{'a': [1,2,3]}``.

But in the code above, since it checks if ``v`` is a dict, then it's not a list, so it would add the new_key with the list as value. So that's handled.

Wait, but what if a value is a list containing dicts? Well, according to the requirements, the function should not flatten lists. So the code treats the list as a non-dict value, so the entire list is added as-is. For example, input ``{'a': [{'b': 1}]}`` would result in ``{'a': [{'b': 1}]}``.

Yes, that's correct.

Now, testing the code.

Let me try the example input:

Input: ``{"a": 1, "b": {"c": 2, "d": {"e": 3}}}``

The code would process:

Start with parent_key empty.

For key 'a', new_key is 'a', value 1 added to result.

For key 'b', value is a dict. So call ``_flatten`` on that dict with parent_key 'b'.

Inside that, for key 'c', new_key is 'b.c', value 2 added.

For key 'd', value is a dict. Call ``_flatten`` with parent_key 'b.d'.

Inside that, key 'e', new_key is 'b.d.e', value 3 added.

So the output is correct.

Another test case: input `{'a': {'b': {'c': {}}}}`. The innermost dict is empty. So during processing, when processing 'c' which has an empty dict, nothing is added. So the output would be empty? Or would the code add the key 'a.b.c' with value `{}`? Let's see. The value is a dict (empty), so the code would call `_flatten` on it. Then, in the loop over items, there are no items. So nothing is added. Therefore, the output would be empty. But the original input has two nested empty dicts. So the output is empty. Which is correct according to the problem's requirements? Well, the problem says to handle empty nested dictionaries. But since there's no value to output, they are just omitted. So that's acceptable.

Another test case: input `{'a': [1, 2, 3]}`. The code would add 'a' with value [1,2,3], which is correct.

Another test case: input `{'a': {'b': {'c': 4}}, 'd': [5,6,7]}`. The output should be `{'a.b.c': 4, 'a.d': [5,6,7]}`.

Yes, the code would handle that.

Now, what about if the input is not a dictionary? Well, the problem states that the input is a dictionary, so we can assume that the function is called with a valid dictionary.

But what if the input is None? Well, the problem doesn't specify, but perhaps the function should handle it gracefully. However, the problem's requirements don't mention handling invalid inputs, so we can assume that the input is a valid dictionary.

Another edge case: a single-level dict. Like `{'a': 1, 'b': 2}` becomes `{'a':1, 'b':2}`.

Another edge case: nested dicts with multiple levels. For example, `{'a': {'b': {'c': {'d': 5}}}}` should become `{'a.b.c.d': 5}`.

The code would handle that recursively.

Now, what about keys that contain dots in the original dictionary? For example, if the input is `{'a.b': {'c': 2}}`, the code would generate the key 'a.b.c'. Which is correct, as the original key 'a.b' is part of the flattened key.

So the code seems to handle all the cases. Now, let me check for any possible issues.

What if the input is an empty dictionary? Then the result is also empty, which is correct.

What if a value is a list containing a dictionary? For example, `{'a': [{'b': 1}]}`. The code would add 'a' with the list as the value, which is correct.

So the code seems to meet all the requirements. Now, let me write the function as described.

</think>

To solve the problem of flattening a nested dictionary with dot-separated keys, we can use a recursive approach. The core idea is to traverse the dictionary and, for each key-value pair, either:

- Recurse into the value if it is a dictionary (with the current key as a prefix), or
- Add the key-value pair to the result with the full key path constructed using dot notation.

This approach ensures that all levels of nesting are handled, and the output dictionary is built incrementally.

? Key Features of the Solution

- ****Handles arbitrary nesting depth****

The recursive function `_flatten` ensures that all levels of nested dictionaries are processed.


```
- **Preserves lists as-is**
If a value is a list, it is added directly to the result without further processing, as per
the requirement.
- **Ignores empty nested dictionaries**
If a dictionary is empty, it contributes nothing
=====
JUDGE REVIEW - ITERATION 2
=====
FEEDBACK:
<think>
Okay, I need to write a Python function called `flatten_dict` that takes a nested dictionary
and returns a flattened version where the keys are joined with dots. Let me think about how to
approach this.
First, the example given is `{ "a": 1, "b": { "c": 2, "d": { "e": 3 } } }` which becomes `{ "a": 1,
"b.c": 2, "b.d.e": 3 }`. So for each nested level, we prepend the parent key with a dot.
The function needs to handle arbitrary nesting. So recursion might be a good approach here.
For each key in the input dictionary, if the value is another dictionary, we recursively
flatten it. But how to accumulate the keys?
Let me think of a helper function that takes the current dictionary, a parent key, and
accumulates into the result. For example, for each key-value pair in the current dict, if the
value is a dict, we call the helper again with the parent key being the current key appended
to the existing parent. Otherwise, we add the combined key to the result.
Wait, but how to start? The initial call would have an empty parent key. Let me outline steps:
Initialize a result dictionary. For each key, value in the input dict:
- If the value is a dict, then recursively process that value, passing the current key as the
parent. Then merge the returned dict into the result.
- Else, add the key (with parent prefix) to the result.
But how to handle the parent key during recursion. Maybe a helper function that takes the
current dictionary and the parent key.
So the main function would call this helper, starting with an empty parent key.
Let me sketch this:
```python
def flatten_dict(d):
def _flatten(current, parent_key, result):
for k, v in current.items():
new_key = parent_key + '.' + k if parent_key else k
if isinstance(v, dict):
_flatten(v, new_key, result)
else:
result[new_key] = v
result = {}
_flatten(d, '', result)
return result
```

[Output continues below...]

...

Wait, but in the example, for the input ``{"a": 1, "b": {"c": 2, "d": {"e": 3}}}``:  
Processing 'a' with parent\_key empty: new\_key is 'a', value is 1, so `result['a'] = 1`.  
Processing 'b' with parent\_key empty: new\_key is 'b'. Since value is a dict, call `_flatten` on that dict with parent\_key 'b'.  
Inside that, for key 'c', new\_key is 'b.c', value 2. Added to result.  
Then key 'd', which is a dict. So call `_flatten` with parent\_key 'b.d'. Inside that, key 'e' gives new\_key 'b.d.e', value 3. So added to result.  
Yes, that gives the correct output.  
But wait, what about if a key is added with a parent key that's part of another key? For example, if there's a key 'a.b' and also a key 'a' with a nested 'b'? But in the problem statement, the input is a nested dict, so that scenario would not happen naturally. But the function is supposed to handle that? Well, in the output, the keys are just strings, and if two different paths result in the same key, the later one would overwrite. But that's up to the user to handle.  
Now, what about empty nested dictionaries? For example, if a key has an empty dict as value. Let's say input is ``{'a': {}}``. The function would process that. Let me see: for each key in the empty dict, there are none. So the `_flatten` function would loop zero times, and the result would have nothing. So the output would be empty? But according to the requirements, handle empty nested dictionaries. So perhaps, if a dict is empty, we should not include it. But since there are no key-value pairs, they don't contribute to the result. So that's correct.  
Another case: if a value is a list. According to the requirements, lists should be kept as-is. So if a key's value is a list, we just add it to the result with the current key. For example, if input is ``{'a': [1,2,3]}``, then output is ``{'a': [1,2,3]}``.  
But in the code above, since it checks if ``v`` is a dict, then it's not a list, so it would add the new\_key with the list as value. So that's handled.  
Wait, but what if a value is a list containing dicts? Well, according to the requirements, the function should not flatten lists. So the code treats the list as a non-dict value, so the entire list is added as-is. For example, input ``{'a': [{'b': 1}]}`` would result in ``{'a': [{'b': 1}]}``.  
Yes, that's correct.  
Now, testing the code.  
Let me try the example input:  
Input: ``{"a": 1, "b": {"c": 2, "d": {"e": 3}}}``  
The code would process:  
Start with parent\_key empty.  
For key 'a', new\_key is 'a', value 1 added to result.  
For key 'b', value is a dict. So call `_flatten` on that dict with parent\_key 'b'.  
Inside that, for key 'c', new\_key is 'b.c', value 2 added.  
For key 'd', value is a dict. Call `_flatten` with parent\_key 'b.d'.  
Inside that, key 'e', new\_key is 'b.d.e', value 3 added.

[Output continues below...]

So the output is correct.

Another test case: input ``{'a': {'b': {'c': {}}}``. The innermost dict is empty. So during processing, when processing 'c' which has an empty dict, nothing is added. So the output would be empty? Or would the code add the key 'a.b.c' with value ``{}``? Let's see. The value is a dict (empty), so the code would call ``_flatten`` on it. Then, in the loop over items, there are no items. So nothing is added. Therefore, the output would be empty. But the original input has two nested empty dicts. So the output is empty. Which is correct according to the problem's requirements? Well, the problem says to handle empty nested dictionaries. But since there's no value to output, they are just omitted. So that's acceptable.

Another test case: input ``{'a': [1, 2, 3]}``. The code would add 'a' with value [1,2,3], which is correct.

Another test case: input ``{'a': {'b': {'c': 4}}, 'd': [5,6,7]}``. The output should be ``{'a.b.c': 4, 'a.d': [5,6,7]}``.

Yes, the code would handle that.

Now, what about if the input is not a dictionary? Well, the problem states that the input is a dictionary, so we can assume that the function is called with a valid dictionary.

But what if the input is None? Well, the problem doesn't specify, but perhaps the function should handle it gracefully. However, the problem's requirements don't mention handling invalid inputs, so we can assume that the input is a valid dictionary.

Another edge case: a single-level dict. Like ``{'a': 1, 'b': 2}`` becomes ``{'a':1, 'b':2}``.

Another edge case: nested dicts with multiple levels. For example, ``{'a': {'b': {'c': {'d': 5}}}`` should become ``{'a.b.c.d': 5}``.

The code would handle that recursively.

Now, what about keys that contain dots in the original dictionary? For example, if the input is ``{'a.b': {'c': 2}}``, the code would generate the key 'a.b.c'. Which is correct, as the original key 'a.b' is part of the flattened key.

So the code seems to handle all the cases. Now, let me check for any possible issues.

What if the input is an empty dictionary? Then the result is also empty, which is correct.

What if a value is a list containing a dictionary? For example, ``{'a': [{'b': 1}]}``. The code would add 'a' with the list as the value, which is correct.

So the code seems to meet all the requirements. Now, let me write the function as described.

</think>

LGTM

Judge approved the code at iteration 2!

## Code

```
import re
Extract only the Python code from the final_code string
code_match = re.search(r"```python\n(?:.*?)```", result_2["final_code"], re.DOTALL)
if code_match:
 executable_code = code_match.group(1)
else:
 executable_code = result_2["final_code"] # Fallback if regex fails, though it might still
 error
Verify the final flatten_dict works
exec(executable_code)
print("Test 1 - Basic nesting:")
print(f" {flatten_dict({'a': 1, 'b': {'c': 2, 'd': {'e': 3}}})}")
print("\nTest 2 - Empty nested dict:")
print(f" {flatten_dict({'a': 1, 'b': {}})}")
print("\nTest 3 - Lists preserved:")
print(f" {flatten_dict({'a': [1, 2, 3], 'b': {'c': [4, 5]}})}")
print("\nTest 4 - Flat dict (no nesting):")
print(f" {flatten_dict({'x': 10, 'y': 20})}")
print("\nTest 5 - Empty dict:")
print(f" {flatten_dict({})}")
```

## Output

```
Test 1 - Basic nesting:
{'a': 1, 'b.c': 2, 'b.d.e': 3}
Test 2 - Empty nested dict:
{'a': 1}
Test 3 - Lists preserved:
{'a': [1, 2, 3], 'b.c': [4, 5]}
Test 4 - Flat dict (no nesting):
{'x': 10, 'y': 20}
Test 5 - Empty dict:
{}
```

Common issues that the Judge typically flags on dictionary flattening functions include: not handling empty nested dictionaries (should they create a key or be skipped?), not handling the case where the input itself is empty, or missing type hints and docstrings. How close was your prediction?

## Key Observations on Reflection

From the two demos above, notice these patterns:

1. First drafts are often functionally correct but miss edge cases. The judge catches what the generator overlooks.
2. Each revision is targeted. The generator does not rewrite from scratch. It surgically addresses the critique.
3. Diminishing returns. The biggest improvements happen in the first revision. By the second or third round, changes become minor (style tweaks, docstring improvements).
4. The LGTM check is important. Without it, the loop would always run for the maximum iterations, even when the code is already good.

This is the same pattern that makes code review at companies like Google effective. The first review catches the big issues. Subsequent reviews catch progressively smaller ones.

## Memory Systems: Conceptual Deep Dive

[TIME: 1:25 to 1:50]

# Memory Systems for Agents

Here is a fundamental problem with LLM agents: by default, they are stateless. Every time you send a new message, the model has no recollection of previous interactions (unless you explicitly include them in the prompt). This is what we call the goldfish problem, meaning the agent forgets everything after each turn.

To build agents that can work on long tasks, learn from past interactions, and maintain context, we need memory systems.

There are three types of memory that matter for agents:

Let us examine each one.

## Short-Term Memory: The Context Window

Short-term memory is the simplest form. It is the context window of the LLM itself. Everything in the current prompt (system message, conversation history, tool results) is the agents short-term memory.

Analogy: Teslas Real-Time Sensor Buffer

Teslas Autopilot processes data from cameras, radar, and ultrasonic sensors in real-time. At any moment, the system has a buffer of the most recent sensor readings. It does not store every frame from every drive. It works with what is in the current buffer.

Similarly, an LLM agents short-term memory is limited to what fits in the context window (e.g., 1 million tokens for Gemini 2.0, 128K for GPT-4). Once the conversation exceeds this limit, the oldest information gets truncated or must be summarized.

The trade-off: Larger context windows mean more short-term memory, but also higher latency and cost. This is why companies like Google and Anthropic are racing to expand context windows. A 1 million token context window is essentially giving the agent a much larger working memory.

...

Context Window = [System Prompt] + [Conversation History] + [Current Query]

Available space for memory = Total Window - System Prompt - Current Query

...

## Episodic Memory: Conversation History

Episodic memory stores the record of past interactions. Unlike short-term memory, it persists between sessions. When you come back to ChatGPT the next day and it remembers your previous conversation, that is episodic memory.

Analogy: Salesforce CRM Storing Customer Interactions

Salesforce CRM keeps a timeline of every interaction with a customer: emails, calls, meetings, support tickets. When a sales rep picks up a conversation, they can scroll back through the history to understand

context.

For agents, episodic memory works the same way:

Implementation approaches:

- Full history: Store every message. Simple but does not scale.
- Summarized history: Periodically summarize older conversations. Saves tokens.
- Windowed history: Keep only the last N messages. Lossy but efficient.

## Long-Term Memory: Vector Stores

Long-term memory is the most powerful and complex type. It uses a vector store (like ChromaDB, Pinecone, or FAISS) to store information as embeddings, which are numerical representations of text that capture semantic meaning.

Analogy: Spotify's Recommendation Index

Spotify does not just store songs as audio files. It creates numerical representations (embeddings) of each song that capture features like tempo, mood, genre, and instrumentation. When you ask for songs like this one, Spotify does not compare audio files. It finds songs whose embeddings are close to the current songs embedding in vector space.

Long-term memory for agents works identically:

1. Store: Convert text (documents, past conversations, knowledge) into embeddings
2. Retrieve: When the agent needs information, convert the query into an embedding and find the most similar stored embeddings
3. Use: Inject the retrieved information into the prompt as context

## Memory Retrieval and Relevance Scoring

The key mechanism that makes long-term memory work is similarity search. When the agent needs to recall something, it computes how similar the current query is to each stored memory.

The most common similarity metric is cosine similarity:

Where  $\vec{A}$  and  $\vec{B}$  are embedding vectors. The result ranges from  $-1$  (opposite) to  $1$  (identical).

Here is pseudocode showing how a memory-augmented agent retrieves relevant information:

## Code

```

PSEUDOCODE: Memory Retrieval System
This is illustrative and not meant to be executed
class AgentMemory:
 """
 Manages long-term memory for an agent using a vector store.
 """
 def __init__(self, embedding_model, vector_store):
 self.embedding_model = embedding_model # e.g., Google text-embedding-004
 self.vector_store = vector_store # e.g., ChromaDB, FAISS, Pinecone
 def store_memory(self, text: str, metadata: dict = None):
 """Store a piece of information in long-term memory."""
 # Convert text to embedding vector (e.g., 768-dimensional float array)
 embedding = self.embedding_model.embed(text)
 # Store in vector database with metadata for filtering
 self.vector_store.add(
 embedding=embedding,
 text=text,
 metadata=metadata # e.g., {"timestamp": "2024-01-15", "source": "user"}
)
 def retrieve_relevant_memories(self, query: str, top_k: int = 3) -> list:
 """Retrieve the most relevant memories for a given query."""
 # Convert query to embedding
 query_embedding = self.embedding_model.embed(query)
 # Search vector store using cosine similarity
 results = self.vector_store.similarity_search(
 query_embedding=query_embedding,
 top_k=top_k
)
 # Results arrive sorted by similarity score (highest first)
 # Example results:
 # [("User prefers Python over Java", 0.92),
 # ("User is working on a web scraping project", 0.87),
 # ("User's favorite food is pizza", 0.12)]
 # Filter by relevance threshold to avoid injecting noise
 relevant = [text for text, score in results if score > 0.5]
 return relevant
 print("Pseudocode: Memory retrieval system architecture")

```



## Output

```
Pseudocode: Memory retrieval system architecture
```

## How These Memory Types Work Together

In a production agent, all three memory types work in concert. Here is a pseudocode example showing a complete memory-augmented agent:

## Code

```

PSEUDOCODE: Complete Memory-Augmented Agent
This is illustrative and not meant to be executed
class MemoryAugmentedAgent:
 """
 An agent that uses all three memory types:
 short-term (context window), episodic (conversation history),
 and long-term (vector store).
 """
 def __init__(self, llm, memory: "AgentMemory"):
 self.llm = llm
 self.memory = memory
 self.conversation_history = [] # Episodic memory (in-session)
 def respond(self, user_message: str) -> str:
 # 1. LONG-TERM MEMORY: Retrieve relevant past knowledge
 relevant_memories = self.memory.retrieve_relevant_memories(
 query=user_message,
 top_k=3
)
 # 2. EPISODIC MEMORY: Include recent conversation turns
 recent_history = self.conversation_history[-10:]
 # 3. SHORT-TERM MEMORY: Everything assembled into the prompt
 # below constitutes the agent's working memory
 prompt = (
 "You are a helpful assistant with access to the following context:\n\n"
 "RELEVANT KNOWLEDGE (from past interactions):\n"
 f"{relevant_memories}\n\n"
 "RECENT CONVERSATION:\n"
 f"{recent_history}\n\n"
 "USER MESSAGE:\n"
 f"{user_message}\n\n"
 "Respond helpfully, using the provided context when relevant."
)
 # Generate response
 response = self.llm.generate(prompt)
 # Update episodic memory for this session
 self.conversation_history.append(f"User: {user_message}")
 self.conversation_history.append(f"Agent: {response}")
 # Persist to long-term memory for future sessions
 self.memory.store_memory(
 text=f"User asked: {user_message}. Agent responded: {response}",
 metadata={"type": "conversation", "timestamp": "2024-07-01"}
)
 return response
print("Pseudocode: Memory-augmented agent architecture")

```

## Output

Pseudocode: Memory-augmented agent architecture

## Real-World Memory Architectures

Different companies use these memory types in different proportions depending on their use case:

### QUIZ 5

Match each scenario to the PRIMARY memory type it relies on. Type your answers in chat as three pairs (e.g., 1-A, 2-B, 3-C).

Scenarios:

1. A coding assistant that remembers you prefer TypeScript over JavaScript across sessions
2. A chatbot that keeps track of what you have said earlier in the current conversation
3. A research agent that searches through 10,000 stored research papers to find relevant ones

Memory Types:

- A. Short-term Memory (context window)
- B. Episodic Memory (conversation history)
- C. Long-term Memory (vector store)

Answers:

1. B (Episodic Memory): Remembering user preferences across sessions is episodic memory. The preference was expressed in a past conversation and needs to persist.
2. A (Short-term Memory): Tracking what was said earlier in the current conversation is handled by the context window.
3. C (Long-term Memory): Searching through thousands of documents for relevant information is exactly what vector store retrieval does.

## Summary and Wrap-Up

[TIME: 1:50 to 2:00]

## Summary: Three Pillars of Advanced Agent Design

Let us recap what we covered today.

### 1. ReAct (Reason + Act)

- \* Agents interleave thinking with tool use in a loop: Thought, Action, Observation
- \* Superior to pure Chain-of-Thought for tasks requiring external information

- \* Adaptive: each step is informed by the result of the previous step
- \* We built an e-commerce support agent with three custom tools and saw it handle single-hop, multi-hop, and edge-case queries
- \* Best suited for research tasks, multi-hop question answering, and dynamic problem solving

## 2. Reflection (Judge-Revise)

- \* Agents evaluate and improve their own outputs through a critique loop
- \* The Generator produces, the Judge critiques, and the Generator revises
- \* We built this with the raw Gemini SDK: three simple functions (generate, judge, revise) composed into a loop
- \* We verified the generated code actually runs correctly after reflection
- \* Most effective when errors are the kind that become obvious upon review
- \* Diminishing returns after 2 to 3 iterations in most cases

## 3. Memory Systems

- \* Short-term memory: Context window. Limited, fast, ephemeral. (Tesla sensor buffer analogy)
- \* Episodic memory: Conversation history. Medium-term, tracks interaction patterns. (Salesforce CRM analogy)
- \* Long-term memory: Vector stores. Persistent, scalable, requires embedding and retrieval. (Spotify recommendation index analogy)
- \* Relevance scoring via cosine similarity determines which memories get retrieved
- \* Production agents typically combine all three memory types

## How These Patterns Combine

In production systems, these patterns are rarely used in isolation:

- \* ReAct + Memory: An agent that reasons and acts while pulling relevant context from long-term memory. Example: a customer support agent that searches the knowledge base (ReAct) while remembering the customers past tickets (Memory).
- \* ReAct + Reflection: An agent that takes actions in the world and then reflects on whether its approach is working. Example: an autonomous coding agent that writes code (ReAct with file tools), tests it, and then reflects on test failures to revise its approach.
- \* All Three Combined: The most capable agents (like advanced versions of Devin or AutoGPT) use all three patterns. They reason and act with tools, maintain memory across long tasks, and reflect on their progress to self-correct.

Understanding these patterns gives you the architectural vocabulary to design agents that go far beyond simple prompt-and-response interactions.